PROF. DR.-ING. MARK SCHUTERA

_SUPERVISED LEARNING

UNFINISHED LECTURE NOTES

Copyright © 2026 Prof. Dr.-Ing. Mark Schutera

PUBLISHED BY UNFINISHED LECTURE NOTES

Combobulated with the help of multiple large language model driven tools. Licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("CC BY-NC-SA 4.0"). You may not use this file for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original You must obtain explicit permission from the author for uses beyond those permitted by this license. To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/. Unless required by applicable law or agreed to in writing, distributed material is provided on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for details.

Last updated, April 2026 — v0.0.9

"IN MY DEFENSE, I WAS LEFT UNSUPERVISED".

DER ZAUBERLEHRLING

Contents

I Geometry of Pattern 11

II Representing Pattern 13

Dimensionality Reduction 15

Variational Inference 35

Index 59

Introduction

Supervised learning is limited to tasks where we have labeled data and clear objectives. Beyond this limited context of learning, we fall back to the pattern in the data itself.

We explore how pattern emerges, from other sources of information beyond explicit labels. We will see how pattern can be discovered in data, represented, leveraged and utilized. In that sense this course is about any form of learning that is not supervised learning.

Each lecture follows a consistent pattern. To start of the contents of the lecture will be motivated by a narrative, a theoretical primer, and some sort of hands-on experience to build intuition. Then we will dive into the, theory and algorithms, then reflection. Theory sessions are 45 minutes, followed by 45 minutes of coding practice to enforce reflection and deepen understanding of the concepts.

A Philosophy of Education

The beautiful risk of education is that it is an encounter. I am trying to model a way of thinking, and reasoning, then stepping back. The rest is yours.

- **Start with why.** A method understood only procedurally is a method you will misapply when it matters. Before learning how something works, understand why it was built, what it replaced, and where it fails. The mechanics then follow from necessity, not memorization.
- **Build taste alongside competence.** You need a sense of what beautiful is, you need to train your aesthetics before you can reliably produce it. This develops by studying what others have built, making it your own, then making your own, and honestly evaluating the gap between them. Taste is the compass. Skill is the engine. You need both, in that order.
- **Failure is the designed medium.** Each attempt teaches what a clean run cannot. The insight compounds across tries, not from a single correct execution. Those who never start failing will fail.
- **Work in the open.** Showing half-finished thinking, naming mistakes as they happen, is a professional skill. We practice it here. It is how knowledge actually moves between people. And it is how knowledge moves towards you, at unprecedented speed.
- **The fallacy of mediocracy.** Iteration and failure are the method, not the excuse for settling. Do not confuse velocity with quality. Whatever you learn, whatever you build: make it good.
- **Outcomes emerge. They are not delivered.** Judgment, taste, the capacity to sit with uncertainty: these are not course outputs. They are what you build from the encounter. I can create the conditions. The rest depends on what you do with them.

Part I

Geometry of Pattern

Part II

Representing Pattern

Dimensionality Reduction

Folding and Unfolding Space.

The Why

Dimensionality reduction finds those factors and discards the rest.

Real-world data $\mathbf{X} \in \mathbb{R}^{n \times d}$ is often high-dimensional: images have millions of pixels, genomes have billions of base pairs, text embeddings have thousands of dimensions. Every clustering method in Part I takes such a feature space as given and works within it. But while dimensionality comes at a cost and degrades our approaches, dimensionality carries information. The question then becomes whether a more compact representation could carry the same information.

MNIST handwritten digits , for example, live in $\mathbb{R}^{784}$ (28×28 pixels), but vary along a handful of latent factors such as shapes or geometric primitives. The intrinsic dimensionality is estimated at roughly $k \approx 12$ to 14.

Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791. https://scholar.google.com/scholar?q=LeCun+Bottou+Bengio+Haffner+1998+gradient-based+learning+document+recognition

Dimensionality has a price. In low dimensions, some points are close and others are far away; that contrast is what makes clustering work. As d grows, the contrast disappears: all pairwise distances converge toward the same value.

Computational cost grows with d, too. Distance computations scale as $O(nd)$, covariance matrices require $O(d^2)$ storage, and eigendecompositions cost $O(d^3)$. Reducing from d to k before fitting a downstream model can turn an intractable pipeline into a practical one.

Expressiveness of the feature space. Original features are often redundant or entangled. If two features $x^{(i)}$ and $x^{(j)}$ are highly correlated ($|\rho_{ij}| \approx 1$), they carry nearly the same information; a good representation replaces them with a single component that captures their shared variance.

Interpretability benefits directly. Humans usually usually do not think and reason in high-dimensional vector spaces; we prefer

low-dimensional, disentangled representations that align with our concepts of space.

In order to move from high-dimensional observations to compact, signal-carrying representations we have good reason to find ways to,

- find a mapping that retains the pattern that carries the signal.
- construct proxy dimensions that are not limited to the original features, so that hidden patterns become apparent.
- understand what different notions of preserving pattern imply, and choose the right one for a given task.

Hands-On Experience

CONSIDER the canonical six points, but now arranged so that $x^{(1)}$ varies only slightly while $x^{(2)}$ carries nearly all the spread.

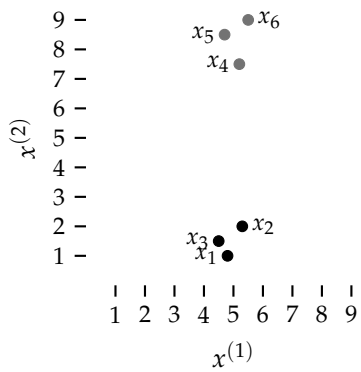


Figure 1: The canonical six points, repositioned so that $x^{(1)}$ barely varies while $x^{(2)}$ carries both the cluster separation and the within-cluster spread. Two features describe each point, but the data is essentially one-dimensional.

THE TWO CLUSTERS are clearly visible, and one feature carries almost all the signal. The first feature $x^{(1)}$ hovers around 5 for every point; dropping it loses almost nothing. The second feature $x^{(2)}$ alone separates the two groups and captures the within-cluster variation.

- If you had to keep only one of the two features, which would you choose?
- How much information would you lose?
- And could you recover the two clusters from that single feature alone?

THE METHODS IN THIS CHAPTER AUTOMATE THAT JUDGMENT OF DIMENSIONALITY REDUCTION. What is the most efficient mapping you can find for the MNIST dataset, using only geometric primitives such as circles, lines so that each class is well-separated?

HAND-CRAFTED FEATURE ENGINEERING is dimensionality reduction by another name. When a domain expert selects "age" and "income" from a database with hundreds of columns, or computes edge gradients from raw pixels, they are projecting from $\mathbb{R}^d$ to $\mathbb{R}^k$ using human genius.

A WELL-CHOSEN MAPPING $f\colon \mathbb{R}^d \to \mathbb{R}^k$ disentangles latent factors, yielding coordinates that are more discriminative for downstream tasks than the originals.

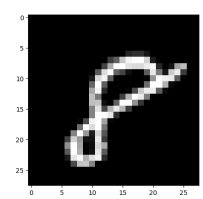


Figure 2: A single MNIST digit its meaningful variation spans few dimensions.

THE LEARNING OBJECTIVES of this lecture:

- Understand naive feature selection based on variance and its limitations.
- Understand the PCA objective and its connection to variance maximization and the singular value decomposition.
- Apply t-SNE and UMAP for visualization and avoid common misinterpretations of their output.
- Choose appropriate dimensionality reduction methods based on linearity, dataset scale, and whether the goal is preprocessing or visualization.

The Curse of Dimensionality, Revisited

THE KEY OBSERVATION is that high d is only fatal when all d dimensions carry independent information. In practice they rarely do. Features are correlated, redundant, or simply noisy. The effective degrees of freedom, the intrinsic dimensionality k, is often far smaller than d.

REDUCING d TO k REVERSES THE CURSE. The spacing, the distance concentration, the density collapse all depend on the dimension of the space the data actually lives in. If we project from $\mathbb{R}^d$ down to $\mathbb{R}^k$, the clustering methods from Part I regain their footing: distances become discriminative again, ε-neighbourhoods fill up, and centroids sit among their members.

THE QUESTION IS HOW TO FIND THE RIGHT k DIRECTIONS. Picking original features by hand (as in variance-based selection) works when the axes happen to align with the signal. When they do not, we need methods that construct new directions from the data itself. That is what PCA, and later t-SNE and UMAP, will do.

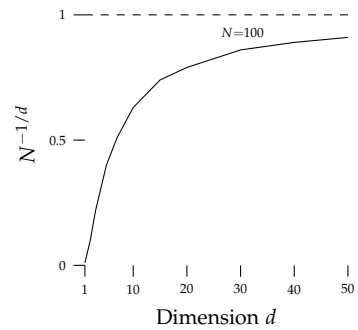


Figure 3: Mean nearest-neighbour spacing $d_{\mathrm{NN}} \propto N^{-1/d}$ for N=100 points. By d=50 every point is nearly as far from its neighbour as from the boundary. But if the data only occupies $k \ll d$ effective dimensions, the curse applies to k, not d.

THE BLESSING OF DIMENSIONALITY. The Johnson-Lindenstrauss lemma shows that n points in high dimensions can be projected to $O(\log n/\epsilon^2)$ dimensions while preserving pairwise distances within $(1\pm\epsilon)$. Even random projections work.

W. B. Johnson and J. Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984. https://scholar.google.com/scholar?q=Johnson+Lindenstrauss+1984+extensions+lipschitz+mappings+hilbert+space

A Naïve Baseline: Variance-Based Feature Selection

WHEN AXES ALIGN WITH THE SIGNAL consider the most straightforward way to reduce dimensions: measure the variance of each original feature independently and keep only those with the highest variance.
Given data $X \in \mathbb{R}^{n\times d}$, compute the variance along each coordinate axis:

$$\mathrm{Var}(x^{(j)}) = \frac{1}{n}\sum_{i=1}^{n}\big(x_i^{(j)} - \bar{x}^{(j)}\big)^2, \quad j = 1,\dots,d$$

Sort the features by decreasing variance, keep the top k, and discard the rest. This is pure feature selection: no new axes are created, only existing ones are retained or dropped.

WHEN DOES THIS WORK? If the coordinate axes already align with the directions of meaningful variation, variance-based selection is fast, interpretable, and surprisingly effective. A near-constant feature carries almost no information, and removing it loses little.

FEATURE SELECTION falls into three families: *filter* methods score features independently of any model (e.g. mutual information, variance thresholding), *wrapper* methods evaluate subsets by training a predictor, and *embedded* methods perform selection inside the learning objective (e.g. ℓ_1 regularisation). Wrapper and embedded methods require labels; filters may or may not. Without labels we are limited to unsupervised proxies such as variance.

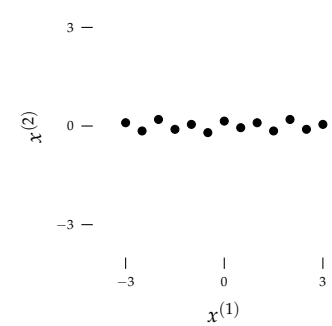


Figure 4: Data spread along $x^{(1)}$ but nearly constant in $x^{(2)}$. Dropping $x^{(2)}$ loses almost nothing.

When does it fail? When the signal does not happen to align with the axis, for example when it runs diagonally across the coordinate axes. In that case, no single original feature captures it well.

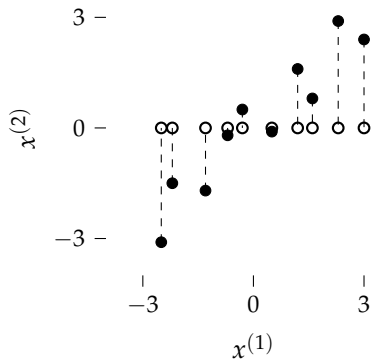


Figure 5: Naïve feature selection: dropping $x^{(2)}$ projects onto the horizontal axis. The dashed errors are large because the data varies along $y{=}x$, not along a coordinate axis. Both features have nearly equal variance, so the naïve ranking cannot decide which to keep. Notice, how in this case both features would still preserver the signal.

Principal Component Analysis

Here both features have nearly the same variance, so variance-based selection cannot decide which to keep. Dropping either axis incurs large reconstruction error. The real pattern lies along the diagonal $y{=}x$, a direction that no original feature represents. When the interesting variation runs between features, we need to construct and map to new dimensions, meaning features. That is exactly what PCA does.

The manifold hypothesis holds that high-dimensional data concentrates on a k-dimensional manifold $\mathcal{M} \subset \mathbb{R}^d$ with $k \ll d$. The goal of dimensionality reduction is to find a coordinate system for $\mathcal{M}$.

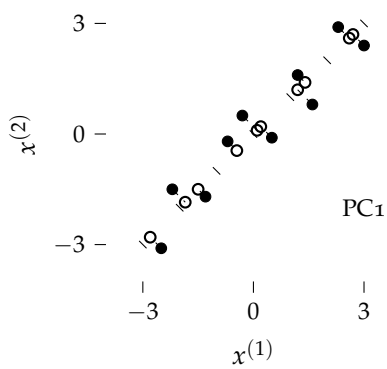


Figure 6: PCA projection onto PC1: The direction of maximum variance runs along $y{=}x$. The reconstruction errors (dashed) are tiny compared to the naïve projection.

Given centred data $X \in \mathbb{R}^{n \times d}$, where rows are samples, PCA finds a k-dimensional subspace, $k < d$, that best represents the data.

PCA finds orthogonal directions of maximum variance in the data.

K. Pearson. On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11):559–572, 1901. DOI: 10.1080/14786440109462720. https://scholar.google.com/scholar?q=Pearson+1901+lines+planes+closest+fit+systems+points; and H. Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441, 1933. DOI: 10.1037/h0071325. https://scholar.

Algorithm 1: PCA via Eigendecomposition

Require: Centred data $X \in \mathbb{R}^{n \times d}$ (each column has zero mean, so that $X^T X/n$ is the covariance matrix; if features have different scales, standardise first by dividing each column by its standard deviation), target dimension k

Ensure: Principal components $W \in \mathbb{R}^{d \times k}$, projected data $Z \in \mathbb{R}^{n \times k}$

1: Compute covariance matrix $\Sigma = X^T X/n$
2: Compute eigendecomposition $\Sigma = V \Lambda V^T$
3: $W \leftarrow$ first k columns of V (sorted by decreasing eigenvalue)
4: $Z \leftarrow XW$ ▷ Project data onto k-dimensional subspace

THE VARIANCE MAXIMIZATION VIEW. Find the direction $w_1 \in \mathbb{R}^d$ with $\|w_1\|=1$ that maximises the variance of projected data:

$$\begin{aligned} w_1 &= \arg\max_{\|w\|=1} \mathrm{Var}(Xw) \\ &= \arg\max_{\|w\|=1} w^T \Sigma\, w, \\ \Sigma &= \tfrac{1}{n} X^T X. \end{aligned}$$

LAGRANGE MULTIPLIERS. Maximising $w^T \Sigma w$ subject to $w^T w = 1$ is a constrained optimisation problem. Introduce a multiplier λ and form the Lagrangian:

$$\mathcal{L}(w, \lambda) = w^T \Sigma\, w \; - \; \lambda\, (w^T w - 1)$$

Differentiating with respect to w and setting the gradient to zero:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w} &= 2\, \Sigma\, w \; - \; 2\, \lambda\, w \\ &\overset{!}{=} \; 0 \end{aligned}$$

which gives the eigenvalue equation:

$$\Sigma\, w = \lambda\, w$$

So w must be an eigenvector of Σ. The projected variance along any such eigenvector equals its eigenvalue:

$$\begin{aligned} w^T \Sigma\, w &= w^T (\lambda\, w) \\ &= \lambda\, w^T w \\ &= \lambda \end{aligned}$$

Maximising the variance therefore means choosing w_1 as the eigenvector with the largest eigenvalue λ_1. The second principal component is the eigenvector with the second-largest eigenvalue, orthogonal to the first, and so on. The first k principal components together define the optimal k-dimensional subspace. $k \leq d$ is a free choice.

ORTHOGONALITY of principal components follows directly: eigenvectors of a symmetric matrix corresponding to distinct eigenvalues are orthogonal.

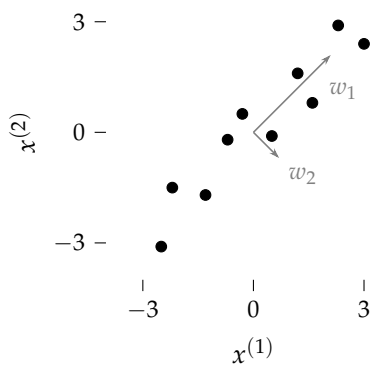


Figure 7: The two eigenvectors of Σ. w_1 points along the direction of maximum variance; w_2 is orthogonal and captures the residual spread.

COMPUTING EIGENVALUES. Eigenvalues satisfy the characteristic equation $\det(\Sigma - \lambda I) = 0$. For a 2×2 covariance matrix $\Sigma = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$:

$$\begin{aligned} \det(\Sigma - \lambda I) &= \det \begin{pmatrix} a-\lambda & b \\ b & c-\lambda \end{pmatrix} \\ &= (a-\lambda)(c-\lambda) - b^2 \\ &= \lambda^2 - (a+c)\,\lambda + (ac - b^2) \\ &\overset{!}{=} 0 \end{aligned}$$

GEOMETRIC INTUITION. Σ stretches space along its eigenvector directions. Subtracting λI weakens every direction by λ. When λ exactly matches the stretch along some eigenvector, $\Sigma - \lambda I$ collapses that direction to zero, the matrix becomes singular, and $\det = 0$. So the characteristic equation asks: at which scaling factors λ does Σ lose a dimension?

Applying the quadratic formula:

$$\begin{aligned} \lambda &= \frac{(a+c) \pm \sqrt{(a+c)^2 - 4(ac - b^2)}}{2} \\ &= \frac{(a+c) \pm \sqrt{(a-c)^2 + 4b^2}}{2} \end{aligned}$$

COMPUTING EIGENVECTORS. For each eigenvalue λ_i, solve $(\Sigma - \lambda_i I)\, w = 0$. From the first row of the 2×2 system:

$$(a - \lambda_i)\, w^{(1)} + b\, w^{(2)} = 0$$

$$w^{(2)} = -\frac{a - \lambda_i}{b}\, w^{(1)}$$

Any non-zero solution gives the direction; normalise to $\|w_i\|=1$ to obtain the principal component.

A SIMPLE DIAGNOSTIC reveals whether the data is compressible. Inspect the eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_d$ of Σ. If most are near zero, the data varies in only a few directions; the remaining dimensions add volume but no information.

The scree plot (Figure 8) visualises exactly this: a steep drop tells you how many components matter.

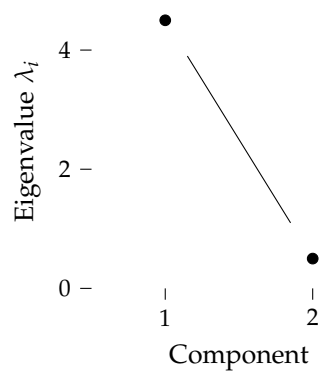


Figure 8: Scree plot for the two-component example (λ_1=4.5, λ_2=0.5). The sharp drop confirms that one direction captures most of the variance.

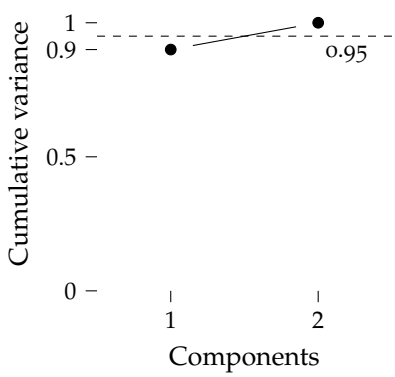


Figure 9: Cumulative explained variance: the fraction of total variance captured by the first k components is $\sum_{i=1}^{k} \lambda_i / \sum_{i=1}^{d} \lambda_i$. Here PC1 alone explains 90%; both components are needed to exceed the rule of thumb 95% threshold (dashed).

Reconstruction Error

SO FAR WE DERIVED PCA FROM VARIANCE MAXIMISATION: find the directions along which the projected data spreads the most. There is a dual view that asks the opposite question: if we compress from d dimensions to k and then decompress back to d, how much signal do we lose?

GIVEN CENTRED DATA $X \in \mathbb{R}^{n \times d}$ and an orthonormal basis $W \in \mathbb{R}^{d \times k}$ ($W^T W = I_k$), each point x_i is projected to $z_i = W^T x_i \in \mathbb{R}^k$ and reconstructed as $\hat{x}_i = W z_i = W W^T x_i$. The reconstruction error is the squared distance between each point and its reconstruction:

$$\begin{aligned}\mathcal{L}(R &= \sum_{i=1}^{n} \|x_i - \hat{x}_i\|^2 \\ &= \sum_{i=1}^{n} \|x_i - W W^T x_i\|^2\end{aligned}$$

THE DASHED ERROR segments are short because PC1 captures the dominant direction of variation, making x and its reconstruction similar.

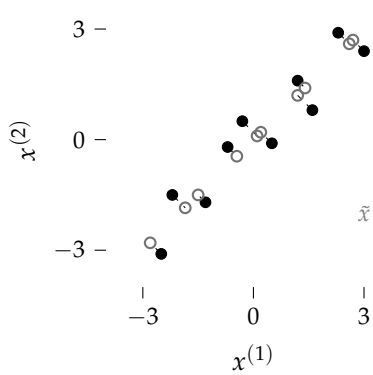


Figure 10: Reconstruction error of PCA with k=1. Each point (filled) is projected onto PC1 (the line y=x) to get its reconstruction (open circle). The dashed segments are the reconstruction errors: the perpendicular distances that PCA minimises. Compare with Figure 5, where projection onto a coordinate axis leaves much larger errors.

FOR $d > 2$, closed-form eigendecompositions do not exist. In practice, use the SVD: $X = U \Sigma V^T$, where the columns of V are the eigenvectors of $X^T X$ with $\lambda_i = \sigma_i^2 / n$. This avoids forming $X^T X$ explicitly and is numerically stable.

Independent Component Analysis

PCA DECORRELATES but does not separate sources. Uncorrelated ($\mathrm{Cov}(s_i, s_j)$=0) is weaker than independent ($p(s_i, s_j) = p(s_i)\, p(s_j)$). Given a linear mixture $X = AS$ of independent signals S, ICA recovers $S = A^{-1} X$ by maximising non-Gaussianity of the components.

t-SNE[1]

[1] L. van der Maaten and G. Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008. https://scholar.google.com/scholar?q=van+der+Maaten+Hinton+2008+visualizing+data+t-SNE

T-SNE. stands for t-distributed Stochastic Neighbour Embedding. It is optimised for one-, two- or three-dimensional visualization. t-SNE preserves local pattern; nearby points in high dimensions stay nearby in the embedding. It does this in four steps.

Algorithm 2: t-SNE. All points are updated simultaneously (batch gradient descent). The KL objective is non-convex, so there is no convergence guarantee; different random initialisations can yield different layouts.

Require: High-dimensional data X, perplexity, target dimension k

1: Compute pairwise similarities in high-D (Gaussian kernel, maps distances to $(0, 1]$):

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2/2\sigma_i^2)}{\sum_{l \neq i} \exp(-\|x_i - x_l\|^2/2\sigma_i^2)}$$

Symmetrize: $p_{ij} = (p_{j|i} + p_{i|j})/2n$

2: Initialize low-D embedding Y

3: **repeat**

4: Compute low-D similarities with Student-t kernel:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

5: Gradient step: minimise

$$\mathrm{KL}(P\|Q) = \sum_{ij} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

6: **until** convergence

PCA AND ICA ARE LINEAR. No rotation of the axes can untangle classes that live on a curved manifold. When the goal is visualization rather than preprocessing, we need methods that can bend the coordinate system to follow the data. This is what t-SNE and UMAP provide.

PERPLEXITY controls the effective number of neighbours each point considers. It sets the bandwidth σ_i of the Gaussian kernel. Typical values: 5 to 50. Higher perplexity means more global pattern is considered.

PERPLEXITY is defined as $2^{H(P_i)}$ where H is the Shannon entropy of the conditional distribution over neighbours. Think of it as the effective number of neighbours.

THE GRADIENT of $\mathrm{KL}(P\|Q)$ with respect to each embedding coordinate is

$$\frac{\partial\, \mathrm{KL}}{\partial\, y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij})\,(y_i - y_j)\,\left(1 + \|y_i - y_j\|^2\right)^{-1}$$

THE KL DIVERGENCE $\mathrm{KL}(P\|Q) = \sum_{ij} p_{ij} \log(p_{ij}/q_{ij})$ measures how much the low-dimensional distribution Q diverges from the high-dimensional distribution P. It is zero when $Q = P$ and asymmetric: misrepresenting a large p_{ij} with a small q_{ij} is penalised heavily, so t-SNE prioritises preserving local neighbourhoods.

Each summand acts as a pairwise force: where $p_{ij} > q_{ij}$ the force is attractive, where $p_{ij} < q_{ij}$ it is repulsive. The heavy tails of the Student-t kernel give distant pairs in Y a larger q_{ij} than a Gaussian would, so the repulsive force saturates. This prevents the crowding

problem where all points collapse into the centre of the map. Gradient descent moves the embedding until local neighbourhoods in Y match those in X.

TRACE the four steps on our canonical six points, mapping from $\mathbb{R}^2$ to $\mathbb{R}^1$ with perplexity=2.

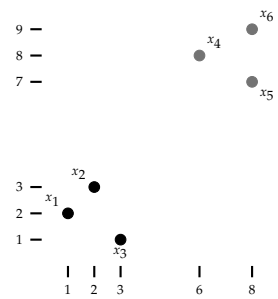
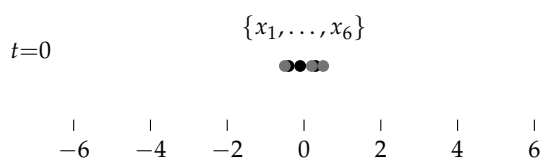
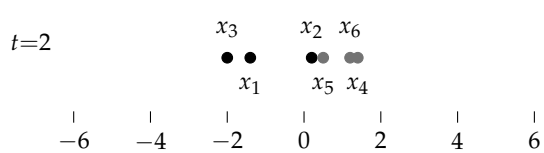


Figure 11: The canonical six points (again)). Cluster A: x_1=(1, 2), x_2=(2, 3), x_3=(3, 1). Cluster B: x_4=(6, 8), x_5=(8, 7), x_6=(8, 9).

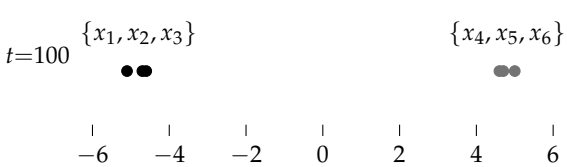
STEP 1: INITIALISE THE EMBEDDING. Each y_i is drawn independently from $\mathcal{N}(0, 10^{-4})$. At t=0 the six points land in near-random order: cluster A (black) and cluster B (grey) are completely intermixed.



STEP 2: COMPUTE HIGH-DIMENSIONAL SIMILARITIES AND BEGIN GRADIENT DESCENT. Here perplexity = 2. Same-cluster pairs get large similarities, cross-cluster pairs near zero; the resulting matrix P is computed once and stays fixed as the target. Gradient descent then pulls high-similarity neighbours together and pushes low-similarity points apart. By t=2 most points drift toward their cluster partners, but x_2; initialised near x_4 (see t=0), needed a few steps for its atractive forces to move through the repulsive forces of the other cluster's points.



STEP 3: CONVERGENCE. At every iteration, q_{ij} is recomputed from the current embedding distances. The forces balance and the embedding stabilises. Within each cluster the ordering mirrors similarity: x_1/x_2 (highest p_{12}=0.098) land closest in A, x_5/x_6 (p_{56}=0.089) in B.



INTERPRETING T-SNE. Cluster sizes and inter-cluster distances are meaningless. t-SNE equalises local densities and preserves only neighbourhood structure. Results vary with the random seed; always run multiple times. Use t-SNE for visualization only, never as preprocessing for downstream models.

UMAP

UMAP stands for Uniform Manifold Approximation and Projection . It has theoretical foundations in Riemannian geometry and algebraic topology.

L. McInnes, J. Healy, and J. Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018. https://arxiv.org/abs/1802.03426

UMAP BRINGS THREE PRACTICAL STRENGTHS.

SCALE. UMAP operates on a sparse k-nearest-neighbour graph instead of all n^2 pairs, giving $O(n \log n)$ complexity in practice, large enough for million-point datasets.

GLOBAL STRUCTURE. UMAP optimises a cross-entropy loss that penalises both pulling true neighbours apart *and* pushing true non-neighbours together, so the embedding preserves large-scale arrangement alongside local clusters.

REPRODUCIBILITY. UMAP initialises the embedding with a spectral embedding of the graph Laplacian, so repeated runs on the same data converge to similar layouts rather than producing arbitrary rotations each time.

Algorithm 3: UMAP. Like t-SNE, all points are updated simultaneously and the objective is non-convex. The spectral initialisation makes results more reproducible than t-SNE's random start, but does not guarantee a global optimum. Key parameters: `n_neighbors` (typically 15–200) controls local versus global structure, analogous to perplexity in t-SNE; `min_dist` (typically 0.1–0.5) controls how tightly points cluster in the embedding.

Require: High-dimensional data X, number of neighbours k, target dimension d, minimum distance δ

1: Build a weighted k-nearest-neighbour graph. For each x_i, set $\rho_i = \min_{j \in \mathrm{NN}(i)} \|x_i - x_j\|$ and choose σ_i so that

$$\sum_{j \in \mathrm{NN}(i)} \exp\bigl(-(\|x_i - x_j\| - \rho_i)/\sigma_i\bigr) = \log_2 k$$

2: Compute directed edge weights:

$$w_{j|i} = \exp\bigl(-\max(0,\ \|x_i - x_j\| - \rho_i)/\sigma_i\bigr)$$

Symmetrise: $w_{ij} = w_{j|i} + w_{i|j} - w_{j|i} \cdot w_{i|j}$

3: Initialise low-D coordinates Y via spectral embedding

4: **repeat**

5: Compute low-D weights:

$$v_{ij} = \bigl(1 + a\,\|y_i - y_j\|^{2b}\bigr)^{-1}$$

6: Gradient step: minimise

$$\mathcal{L} = \sum_{ij} \Bigl[w_{ij} \log \frac{w_{ij}}{v_{ij}} + (1 - w_{ij}) \log \frac{1 - w_{ij}}{1 - v_{ij}} \Bigr]$$

7: **until** convergence

Examples & Exercises

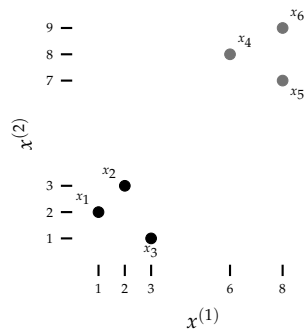


Figure 12: The canonical six points from Chapters 3 and 4. Cluster A: $x_1{=}(1,2)$, $x_2{=}(2,3)$, $x_3{=}(3,1)$. Cluster B: $x_4{=}(6,8)$, $x_5{=}(8,7)$, $x_6{=}(8,9)$.

Computing by hand builds the geometric intuition that plotting the output never can. Step through the arithmetic slowly.

Given the canonical six points $x_1{=}(1,2)$, $x_2{=}(2,3)$, $x_3{=}(3,1)$, $x_4{=}(6,8)$, $x_5{=}(8,7)$, $x_6{=}(8,9)$, centre the data, compute the covariance matrix Σ, find its eigenvalues and eigenvectors, and determine what fraction of variance PC1 explains.

Centering. PCA requires zero-mean data. Compute the sample mean:

$$\begin{aligned}\bar{x} &= \frac{1}{6}\sum_{i=1}^{6} x_i \\ &= \left(\frac{28}{6},\ \frac{30}{6}\right) \\ &= \left(\frac{14}{3},\ 5\right)\end{aligned}$$

Subtract $\bar{x}$ from each point. Here is the full calculation for x_1:

$$\begin{aligned}\tilde{x}_1 &= x_1 - \bar{x} \\ &= \left(1 - \tfrac{14}{3},\ 2 - 5\right) \\ &= \left(-\tfrac{11}{3},\ -3\right)\end{aligned}$$

Compute the remaining five yourself, then verify against the table below.

Point	**Original**	**Centred** $\tilde{x}_i$
x_1	(1, 2)	$(-11/3,\ -3)$
x_2	(2, 3)	$(-8/3,\ -2)$
x_3	(3, 1)	$(-5/3,\ -4)$
x_4	(6, 8)	$(4/3,\ 3)$
x_5	(8, 7)	$(10/3,\ 2)$
x_6	(8, 9)	$(10/3,\ 4)$

Table 1: Centred data. The second coordinate centres cleanly ($\bar{x}^{(2)}{=}5$); the first introduces thirds ($\bar{x}^{(1)}{=}14/3$).

THE COVARIANCE MATRIX. $\Sigma = \tilde{X}^T \tilde{X}/n$. Here is the full calculation for Σ_{11}:

$$\begin{aligned}
\Sigma_{11} &= \frac{1}{6}\sum_{i=1}^{6}\left(\tilde{x}_i^{(1)}\right)^2 \\
&= \frac{1}{6}\left[\frac{121}{9} + \frac{64}{9} + \frac{25}{9} + \frac{16}{9} + \frac{100}{9} + \frac{100}{9}\right] \\
&= \frac{1}{6} \cdot \frac{426}{9} \\
&= \frac{71}{9} \\
&\approx 7.89
\end{aligned}$$

Compute Σ_{12} and Σ_{22} yourself. The full matrix:

$$\Sigma = \begin{pmatrix} 71/9 & 47/6 \\ 47/6 & 29/3 \end{pmatrix} \approx \begin{pmatrix} 7.89 & 7.83 \\ 7.83 & 9.67 \end{pmatrix}$$

$\Sigma_{12} = 47/6 \approx 7.83$: both features are strongly positively correlated, as expected from the scatter plot where both coordinates increase together from cluster A to cluster B.

EIGENVALUES. Apply the formula from the theory section with $a = 71/9$, $b = 47/6$, $c = 29/3$:

$$\begin{aligned}
a + c &= \frac{71}{9} + \frac{29}{3} \\
&= \frac{158}{9} \\
&\approx 17.56
\end{aligned}$$

$$\begin{aligned}
(a-c)^2 + 4b^2 &= \left(\frac{-16}{9}\right)^2 + 4\left(\frac{47}{6}\right)^2 \\
&= \frac{256}{81} + \frac{2209}{9} \\
&= \frac{20137}{81} \\
&\approx 248.6
\end{aligned}$$

So:

$$\begin{aligned}
\lambda &= \frac{17.56 \pm \sqrt{248.6}}{2} \\
&\approx \frac{17.56 \pm 15.77}{2}
\end{aligned}$$

$\lambda_1 \approx 16.66$ and $\lambda_2 \approx 0.89$.

EIGENVECTOR FOR $\lambda_1 \approx 16.66$. Solve $(a - \lambda_1)\, w^{(1)} + b\, w^{(2)} = 0$:

$$\begin{aligned}
(7.89 - 16.66)\, w^{(1)} + 7.83\, w^{(2)} &= 0 \\
-8.77\, w^{(1)} + 7.83\, w^{(2)} &= 0 \\
w^{(2)} &= \frac{8.77}{7.83}\, w^{(1)} \\
&\approx 1.12\, w^{(1)}
\end{aligned}$$

$$\begin{aligned}
\|w_1\| &= \sqrt{1^2 + 1.12^2} \\
&= \sqrt{2.25} \\
&= 1.50, \\
w_1 &\approx \frac{1}{1.50} \begin{pmatrix} 1 \\ 1.12 \end{pmatrix} \\
&\approx \begin{pmatrix} 0.67 \\ 0.75 \end{pmatrix}.
\end{aligned}$$

EIGENVECTOR FOR $\lambda_2 \approx 0.89$. Solve $(a - \lambda_2)\, w^{(1)} + b\, w^{(2)} = 0$:

$$\begin{aligned}
7.00\, w^{(1)} + 7.83\, w^{(2)} &= 0 \\
w^{(2)} &\approx -0.89\, w^{(1)}
\end{aligned}$$

Normalising: $w_2 \approx \begin{pmatrix} 0.75 \\ -0.67 \end{pmatrix}$.

Verify orthogonality: $w_1^T w_2 = (0.67)(0.75) + (0.75)(-0.67) = 0$. ✓

EXPLAINED VARIANCE BY PC1:

$$\begin{aligned}
\frac{\lambda_1}{\lambda_1 + \lambda_2} &\approx \frac{16.66}{17.56} \\
&\approx 95\%
\end{aligned}$$

GEOMETRIC INTERPRETATION. PC1 points roughly along the direction from cluster A (bottom left) to cluster B (top right). The two clusters that are visible in two dimensions can be separated using a single principal component.

PROJECT AND RECONSTRUCT. Using the eigenvectors from the previous exercise, project each of the six centred points onto PC1, reconstruct them in the original space, and compute the reconstruction error.

PROJECTION ONTO PC1. For each centred point $\tilde{x}_i$, the coordinate on PC1 is $z_i = \tilde{x}_i^T w_1$. Here is the full calculation for x_1:

$$\begin{aligned} z_1 &= \tilde{x}_1^T w_1 \\ &= (-3.67)(0.67) + (-3)(0.75) \\ &= -2.46 - 2.25 \\ &= -4.71 \end{aligned}$$

Compute z_2 through z_6 yourself, then verify against the table below.

RECONSTRUCTION. Each projected point maps back to 2D via $\hat{x}_i = z_i \cdot w_1$. For x_1:

$$\begin{aligned} \hat{x}_1 &= -4.71 \cdot \begin{pmatrix} 0.67 \\ 0.75 \end{pmatrix} \\ &= \begin{pmatrix} -3.16 \\ -3.53 \end{pmatrix} \end{aligned}$$

RECONSTRUCTION ERROR:

$$\begin{aligned} \|\tilde{x}_1 - \hat{x}_1\|^2 &= (-3.67 + 3.16)^2 + (-3.00 + 3.53)^2 \\ &= (-0.51)^2 + (0.53)^2 \\ &= 0.26 + 0.28 \\ &= 0.54 \end{aligned}$$

Point	**Centred** $\tilde{x}_i$	z_i	**Reconstructed** $\hat{x}_i$	$\|\tilde{x}_i - \hat{x}_i\|^2$
x_1	$(-3.67, -3)$	-4.71	$(-3.16, -3.53)$	0.54
x_2	$(-2.67, -2)$	-3.29	$(-2.20, -2.47)$	0.44
x_3	$(-1.67, -4)$	-4.12	$(-2.76, -3.09)$	2.02
x_4	$(1.33, 3)$	3.14	$(2.10, 2.36)$	1.01
x_5	$(3.33, 2)$	3.73	$(2.50, 2.80)$	1.33
x_6	$(3.33, 4)$	5.23	$(3.50, 3.92)$	0.04
			Total	**5.38**

Table 2: PCA projection onto PC1 and reconstruction for the canonical six points. The total error $\approx n \cdot \lambda_2 = 6 \times 0.89 = 5.34$ (the small difference is rounding in intermediate steps). Point x_3 has the largest error because it sits furthest from the PC1 direction; x_6 has almost none because it lies nearly on PC1.

NAÏVE FEATURE SELECTION VS PCA. Using the same six centred points, compare the reconstruction error of naïve feature selection with PCA.

DROP $x^{(2)}$ (KEEP $x^{(1)}$). The reconstruction is $\hat{x}_i = (\tilde{x}_i^{(1)}, 0)$. For x_1:

$$\begin{aligned}\|\tilde{x}_1 - \hat{x}_1\|^2 &= (-3.67 - (-3.67))^2 + (-3 - 0)^2 \\ &= 0 + 9 \\ &= 9\end{aligned}$$

DROP $x^{(1)}$ (KEEP $x^{(2)}$). The reconstruction is $\hat{x}_i = (0, \tilde{x}_i^{(2)})$. For x_1:

$$\begin{aligned}\|\tilde{x}_1 - \hat{x}_1\|^2 &= (-3.67 - 0)^2 + (-3 - (-3))^2 \\ &= 13.47 + 0 \\ &= 13.47\end{aligned}$$

Compute the remaining errors yourself, then verify against the table.

Point	**Drop $x^{(2)}$**	**Drop $x^{(1)}$**	**PCA (PC1)**
x_1	9.00	13.44	0.54
x_2	4.00	7.11	0.44
x_3	16.00	2.78	2.02
x_4	9.00	1.78	1.01
x_5	4.00	11.11	1.33
x_6	16.00	11.11	0.04
Total	**58.00**	**47.33**	**5.38**

Table 3: Reconstruction error comparison. The naïve variance ranking correctly picks $x^{(2)}$ ($\mathrm{Var}(x^{(2)}) \approx 9.67 > \mathrm{Var}(x^{(1)}) \approx 7.89$), yet PCA still reduces the error by a factor of ≈ 9.

The naïve approach gets the ranking right: $x^{(2)}$ does carry more variance than $x^{(1)}$. But neither coordinate axis aligns with the direction the data actually varies along. PCA constructs that direction from the data and achieves roughly nine times less error.

CODE is provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

THE CODE EXCERCISES LOAD 1 000 handwritten digits from MNIST: 784 pixels each, ten classes. In code, you will apply three preprocessing stages: discard uninformative pixels via a variance threshold, centre the data by subtracting the per-feature mean (without centring, PC1 aligns with the mean intensity rather than with the dominant direction of variation), and eigendecompose the covariance matrix to project onto the top two components. Then compare the PCA projection with t-SNE (sweeping perplexity) and UMAP (sweeping neighbourhood size).

WHAT TO OBSERVE. PCA preserves global geometry: The ten digit classes form broad, overlapping regions whose distances reflect the original space. Because PCA is linear, it cannot untangle classes that

live on a curved manifold; overlap in the 2D plot is a direct consequence. t-SNE compresses each class into a tight ball, but distances between balls are arbitrary and change with the random seed. UMAP produces similarly tight clusters while retaining more of the global arrangement. Both nonlinear methods are sensitive to their main hyperparameter; the sweep makes this visible.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- What does it mean for data to have low intrinsic dimensionality?
- Why does PCA correspond to an eigenvector decomposition of the covariance matrix?
- What is the difference between an uncorrelated and an independent component?
- Why does t-SNE use a Student-t kernel in the low-dimensional space but a Gaussian in the high-dimensional space?
- When would you prefer UMAP over t-SNE, and when would you use PCA instead of either?
- What does the perplexity parameter actually control, and what happens when it is too low or too high?
- Why should you never use t-SNE or UMAP embeddings as features for a downstream classifier?
- How does the curse of dimensionality motivate dimensionality reduction before running K-Means?

RECAP of Key Concepts:

- PCA finds the orthogonal directions of maximum variance via eigendecomposition of the covariance matrix; SVD is the numerically preferred implementation.
- The explained-variance ratio guides the choice of k; retain enough components to exceed a threshold such as 95%.
- t-SNE and UMAP preserve local neighbourhood patterns for visualization; cluster sizes and between-cluster distances in their output are not interpretable.

- ICA finds statistically independent sources by maximising non-Gaussianity, going beyond the uncorrelated directions found by PCA.
- No non-linear method should be used as a preprocessing step for downstream learning; use PCA or SVD for that purpose.

PCA AND ICA CAN ONLY ROTATE AND SCALE AXES. When the underlying structure is nonlinear, no linear map can recover it. t-SNE and UMAP solve this for visualization, but neither learns a parametric encoder that can be applied to new data or reused as a preprocessing step.

THE FUNDAMENTAL LIMIT of PCA and ICA is linearity; they cannot capture curved structure. t-SNE and UMAP learn nonlinear embeddings, but are designed for visualization, not for learning a reusable mapping.

AUTOENCODERS learn both the compression and the reconstruction from data. The encoder is a neural network that maps x_i to a low-dimensional code z_i; the decoder maps z_i back to $\hat{x}_i$. Variational Autoencoders go one step further: the encoder outputs not a single point but a mean μ and a variance σ^2, turning the code into a distribution from which new data can be sampled.

TEASER. If we replace the fixed eigenvector projection with a learned, nonlinear encoder, can we get a compact representation that generalises to unseen inputs?

FEEDBACK

Variational Inference

EMBEDDINGS AND LATENT FEATURE SPACES.

The Why

THE CENTRAL PROBLEM OF UNSUPERVISED LEARNING is representation learning: finding a mapping from raw observations to a latent feature space in which the underlying factors of variation become explicit. Bengio, Courville, and Vincent[2] argue that the quality of a representation determines the success of any downstream task, and that a good representation disentangles the factors that explain the data.

PLATO'S CAVE ALLEGORY describes prisoners who see only shadows on a wall, never the objects that cast them. The shadows are high dimensional and entangled; the true forms behind them are few and independent. Dimensionality reduction, as we have seen, tries to recover those forms from the shadows. This chapter asks two follow-up questions: can we organise the recovered forms into a structured, reusable feature space, and once we have that space, can we cast new, plausible shadows we have never observed?

Plato. *The Republic*. Penguin Classics, 2007. Book VII, 514a–520a. Originally written c. 380 BCE

PCA GAVE US A FIRST ANSWER: project onto the directions of maximum variance. But PCA is linear; it cannot recover structure that lies on a curved manifold. What we lack is a parametric encoder that can be applied to new, unseen data or reused as a feature extractor on nonlinear feature spaces.

AUTOENCODERS close that gap. Hinton and Salakhutdinov[3] showed that a neural network can learn a nonlinear, low-dimensional embedding z from which a second network reconstructs $\hat{x}$, outperforming PCA on complex data. The encoder $\theta_E \colon \mathbb{R}^d \to \mathbb{R}^k$ and the decoder $\theta_D \colon \mathbb{R}^k \to \mathbb{R}^d$ are trained jointly to minimise reconstruction error $\|x - \theta_D(\theta_E(x))\|^2$. The bottleneck forces the network to discover a compact representation.

STRUCTURE REQUIRES A PROBABILISTIC FORMULATION.

If we replace the deterministic bottleneck with a distribution, and regularise that distribution toward a known prior, the latent space becomes smooth and complete: every region maps to a plausible output, and nearby embeddings correspond to similar inputs. Kingma and Welling[4] formalised this idea as the Variational Autoencoder, deriving a principled training objective from variational inference that

[2] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013. DOI: 10.1109/TPAMI.2013.50. https://scholar.google.com/scholar?q=Bengio+Courville+Vincent+2013+representation+learning+review+new+perspectives

[3] G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006. DOI: 10.1126/science.1127647. https://scholar.google.com/scholar?q=Hinton+Salakhutdinov+2006+reducing+dimensionality+neural+networks

A PRIOR is the distribution we assume over the latent space before seeing any data.

[4] D. P. Kingma and M. Welling. Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*, 2014. https://scholar.google.com/scholar?q=Kingma+Welling+2014+Auto-Encoding+Variational+Bayes

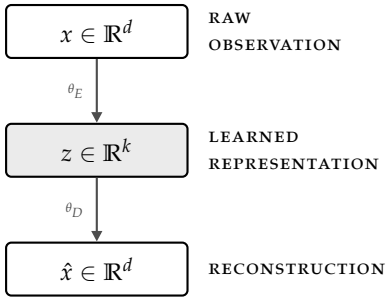


Figure 13: Autoencoder architecture. The encoder θ_E maps the input to a low-dimensional representation; the decoder θ_D reconstructs the input from it. The bottleneck forces the network to discover a compact latent feature space.

balances faithful reconstruction against latent space regularity.

ALL OF THIS MUST HAPPEN WITHOUT LABELS. Supervised learning can organise a feature space by asking "which class does this input belong to?" and adjusting representations until the answer is easy to read off. An unsupervised model has no such signal. Instead, it must derive structure from the data alone, using two complementary pressures: the requirement to reconstruct the input faithfully, and the requirement to keep the latent space close to a known prior. Reconstruction forces the embedding to retain information; the prior forces the embedding to be organised. Neither pressure requires a human to annotate a single example.

IN ORDER TO MOVE FROM RECONSTRUCTION-ONLY EMBEDDINGS TO WELL-ORGANISED LATENT FEATURE SPACES we have good reason to find ways to,

- formulate a learning objective that rewards both faithful reconstruction and a smooth, regular latent space.
- train a model that outputs distributions rather than points, so that the latent space has no undefined gaps.
- control how much structure the latent space carries, and understand how that choice affects the quality of the learned representation.

Hands-On Experience

FORM GROUPS OF TWO: an encoder and a decoder. Prepare five digit cards showing **0**, **1**, **3**, **7**, and **8**. Shuffle them face down on the table.

STEP 1: DESIGN YOUR FEATURES. The encoder picks two properties of digit shapes and writes them down. These are your two latent features $z^{(1)}$ and $z^{(2)}$. For example: $z^{(1)}$ = "number of closed loops" (0 has one, 8 has two, 1 has none) and $z^{(2)}$ = "has a straight vertical stroke" (1 yes, 7 yes, 0 no); make sure to come up with your own The decoder does not see these definitions.

SCORING. After the decoder guesses, flip the card. Correct digit = 0, wrong digit = 1. Sum over all rounds: that is your reconstruction loss.

STEP 2: PLAY. Each round, the encoder flips one card, looks at the digit, and writes two numbers $(z^{(1)}, z^{(2)})$ on a slip of paper. The decoder receives only the slip and guesses which digit it is. Then reveal the card. The decoder now knows: "the embedding $(1, 0)$ meant the digit 3." Round by round, the decoder builds a mental map of what the embeddings mean. When all five cards are done, shuffle and keep going.

STEP 3: OBSERVE. In early rounds, the decoder guesses blindly. With each reveal, the decoder learns what the embeddings mean. At some point the reconstruction loss drops to zero: the decoder has learned the encoder's representation from examples alone.

STEP 4: RECORD AND COMPARE. Plot your five embeddings in the coordinate system below.

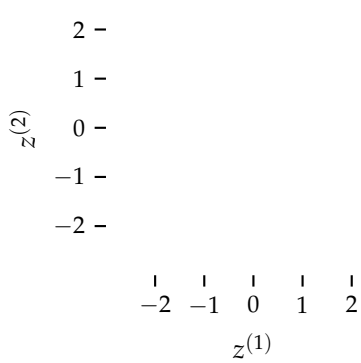


Figure 14: Your latent space. Plot the five digit embeddings at the coordinates $(z^{(1)}, z^{(2)})$ your encoder chose.

STEP 5: SWAP DECODERS. Find another group. Send your decoder to them, take theirs. Play one more pass with the swapped decoder.

- How many rounds did the decoder need before reconstructions became reliable?
- Which digits did the decoder confuse most often? What does that tell you about your feature choice?

The swapped decoder has learned a different codebook: in their group, $(1, 0)$ might have meant "7," in yours it means "3." The reconstruction loss jumps back up. The problem is not that either representation is wrong; both worked within their own group. The problem is that nothing forced the two groups to organise their embeddings the same way.

STEP 6: SHRINK THE BOTTLENECK. Drop to a single latent feature $z^{(1)}$. The encoder must pick the one property that matters most; the decoder has only one number to work with. Play a full pass and record the new reconstruction loss.

STEP 7: DISCRETISE. Go back to two features, but restrict each to binary values (0 or 1). The encoder can no longer spread digits across a continuous range and must find features that cleanly partition the digit set with just two bits.

STEP 8: ADD DIGITS. Keep two binary features, but add the cards **2**, **4**, **5**, and **9**. A representation that separated five digits may collapse when it must also distinguish eight. But the extra classes also expose ad hoc feature choices: properties that happened to work for a small set get overwhelmed, and the encoder is pressured toward genuinely structural features (loops, strokes, intersections) that scale.

STEP 9: SWAP DECODERS AGAIN. Find a different group than in Step 5. Exchange decoders and play one pass with the expanded digit set. Compare the reconstruction loss to the Step 5 swap: did the stress tests in Steps 6 to 8 push both groups toward more compatible representations.

After each decoder swap, collect the chosen latent features from all groups on the board. Are the feature sets converging across the room? Which features survived the stress tests and which were abandoned?

EACH STEP HURTS RECONSTRUCTION but forces the encoder to find features that generalise across inputs rather than overfit to a narrow set. This tension between reconstruction fidelity and generalisation through compression or broader coverage is exactly the trade-off that the VAE formalises. And a trade-off you know from supervised learning as well, overfitting versus generalization.

THIS IS THE REPRESENTATION LEARNING PROBLEM. Every choice of features defines a different geometry: different neighbours, different distances. Within one encoder-decoder pair, reconstruction works. Across pairs, the embeddings are incompatible.

THE FUNDAMENTAL QUESTION is: can we add a shared convention to the latent space, so that the embeddings are not only good for

TWO OBJECTIVES, ONE GAME. Steps 2 to 4 trained the reconstruction part: the decoder learned to recover digits from embeddings. Steps 5 and 9 exposed the missing regularisation: without a shared convention for what the embeddings mean, a new decoder cannot read them. Steps 6 to 8 showed that compression and broader coverage force better features. A VAE adds exactly this second pressure: it forces every encoder to arrange its embeddings according to a shared prior, so that any decoder can use them.

reconstruction but also follow a common structure that any decoder could interpret?

THE LEARNING OBJECTIVES of this lecture:

- The Autoencoder Training Cycle. Architecture, forward pass, reconstruction loss, backpropagation, and what the bottleneck forces the network to learn.
- From Autoencoders to VAEs. Why reconstruction alone leaves the latent space unstructured, and what a probabilistic formulation buys us.
- The Evidence Lower Bound. The training objective that balances faithful reconstruction against latent space regularity.
- The Reparameterisation Trick. How to backpropagate through a stochastic sampling step.
- VAE Training and Bottleneck Design. Practical choices that shape the latent space: loss functions, latent dimensionality, and numerical stability.
- Exploring the Latent Space. Interpolation and generation as tests of representation quality.
- VAE Variants. How they each change one structural assumption.
- Feature Extraction, Transfer, and Distillation. Using the trained encoder as a backbone for downstream tasks.

The Autoencoder

Follows the notion of a bottleneck architecture: an encoder that compresses the input into a low-dimensional embedding, and a decoder that reconstructs the input from that embedding. Due to the bottleneck, the encoder must learn to extract the most salient features of the input, and the decoder must learn to use those features to reconstruct the input as losslessly as possible.

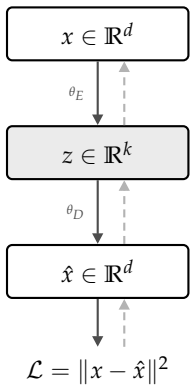


Figure 15: Autoencoder training cycle. Solid arrows: forward pass ($x \to z \to \hat{x} \to \mathcal{L}$). Dashed arrows: gradient flow back through decoder and encoder.

THE ENCODER $\theta_E \colon \mathbb{R}^d \to \mathbb{R}^k$ compresses a d-dimensional input x into a k-dimensional embedding z, with $k \ll d$.

NOTATION. θ_E denotes the encoder; θ_D denotes the decoder. Both are neural networks trained jointly.

THE DECODER $\theta_D \colon \mathbb{R}^k \to \mathbb{R}^d$ takes z and attempts to reconstruct the original input as $\hat{x}$.

THE FORWARD PASS maps a single training example through both networks. Given an input x, the encoder produces the embedding:

$$z = \theta_E(x)$$

The decoder then reconstructs:

$$\begin{aligned} \hat{x} &= \theta_D(z) \\ &= \theta_D(\theta_E(x)) \end{aligned}$$

BACKPROPAGATION is the backbone of supervised learning, and it applies here unchanged. The loss $\mathcal{L}$ depends on θ_D (through $\hat{x}$) and on θ_E (through z, which feeds into $\hat{x}$). The chain rule gives:

$$\frac{\partial \mathcal{L}}{\partial \theta_D} = \frac{\partial \mathcal{L}}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial \theta_D}$$

for the decoder, and

$$\frac{\partial \mathcal{L}}{\partial \theta_E} = \frac{\partial \mathcal{L}}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial z} \cdot \frac{\partial z}{\partial \theta_E}$$

for the encoder. The gradient signal flows from the loss, through the decoder, through z, and into the encoder. Both networks are updated from the same single loss.

THE RECONSTRUCTION LOSS measures how accurate the decoder reconstructs the input. The standard choice is the mean squared error over all dimensions:

$$\begin{aligned} \mathcal{L}(\theta_E, \theta_D; x) &= \|x - \hat{x}\|^2 \\ &= \sum_{j=1}^{d} (x_j - \hat{x}_j)^2 \end{aligned}$$

THE PARAMETER UPDATE applies standard gradient descent (or a variant such as Adam):

$$\theta_E \leftarrow \theta_E - \eta \frac{\partial \mathcal{L}}{\partial \theta_E}$$

$$\theta_D \leftarrow \theta_D - \eta \frac{\partial \mathcal{L}}{\partial \theta_D}$$

where η is the learning rate. One forward pass, one backward pass, one update: that is a single training step. Repeating this over minibatches and epochs, the encoder and decoder co-adapt until the reconstruction error converges.

THE BOTTLENECK. Because $k \ll d$, the embedding z cannot simply copy x. The encoder must decide which aspects of the input to preserve and which to discard. This forced compression is what generalizes the encoder into a feature extractor.[5]

THE TRAINED ENCODER IS A FEATURE EXTRACTOR, OR DIMENSIONALITY REDUCER. Once training converges, the decoder can be discarded. The encoder alone maps any new input x to an embedding z that captures the factors of variation learned during training. This embedding can serve as input to a classifier, a clustering algorithm, or a retrieval system. The autoencoder has learned a representation; the question is whether that representation is generalized.

$$x \in \mathbb{R}^d \xrightarrow{\theta_E} z \in \mathbb{R}^k$$

CONNECTION TO PCA. If the encoder and decoder are both linear (no activation functions) and the loss is MSE, the autoencoder recovers the same subspace as PCA[6]. The k embedding dimensions span the top-k principal component directions. Adding nonlinear activations allows the network to capture structure that PCA cannot: curved manifolds, discrete clusters, hierarchical features.

[5] G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006. DOI: 10.1126/science.1127647. https://scholar.google.com/scholar?q=Hinton+Salakhutdinov+2006+reducing+dimensionality+neural+networks

WHY DEEP (LEARNING)? Each layer applies a linear transformation followed by a nonlinear activation. A single hidden layer can in principle approximate any function, but may need impractically many neurons and is hard to learn. Deeper networks compose simple nonlinearities into increasingly abstract features: the first layer detects edges, the second combines edges into textures, the third recognises parts. Each layer reuses the representations below it, so the network expresses complex structure with far fewer parameters than a shallow one. Hinton and Salakhutdinov showed that a deep autoencoder with the same total capacity as a shallow one produces qualitatively better embeddings, because hierarchical composition is a more efficient way to represent the kind of structure found in real data.

[6] P. Baldi and K. Hornik. Neural networks and principal component analysis: Learning from examples without local minima. *Neural Networks*, 2(1):53–58, 1989. DOI: 10.1016/0893-6080(89)90014-2. https://scholar.google.com/scholar?q=Baldi+Hornik+1989+neural+networks+principal+component+analysis

From Autoencoders to Variational Autoencoders

THE LATENT SPACE HAS NO GEOMETRIC GUARANTEES. The objective only rewards reconstruction; nothing constrains how embeddings are arranged. Nearby points may decode to unrelated outputs, gaps between embeddings are undefined, and the layout can change arbitrarily across training runs. The representation captures enough to reconstruct, but it does not organise the factors of variation in a way that supports interpolation, sampling, or transfer.

STANDARD AUTOENCODERS learn $x \to z \to \hat{x}$, minimising $\|x - \hat{x}\|^2$.

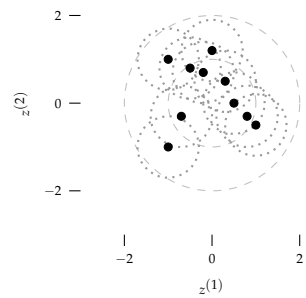
TO FIX THIS, WE ADD A CONSTRAINT. In a plain autoencoder, the encoder's output layer produces a single vector $z \in \mathbb{R}^k$: one fixed point per input. In a Variational Autoencoder, the output layer produces two vectors instead: a mean $\mu \in \mathbb{R}^k$ and a variance $\sigma^2 \in \mathbb{R}^k$. Together they define a Gaussian distribution over the latent space:

$$q_{\theta_E}(z|x) = \mathcal{N}(\mu, \sigma^2)$$

During training, we do not pass μ directly to the decoder. Instead we sample from that distribution:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

So the decoder sees a slightly different z every time, even for the same input.



VAE latent space. Each small circle is one input's encoder distribution $q_{\theta_E}(z|x_i)$; the dot marks its mean μ. The large dashed circles show the prior $p(z) = \mathcal{N}(0, I)$ at 1σ and 2σ. The KL term pushes each small circle toward unit variance and its mean toward the origin, so that the mixture of all encoder distributions approximates the prior.

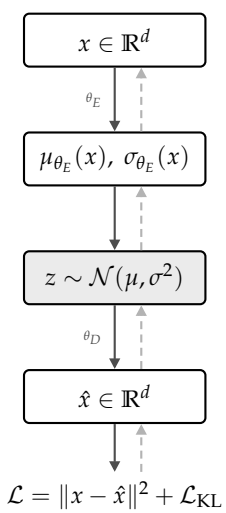
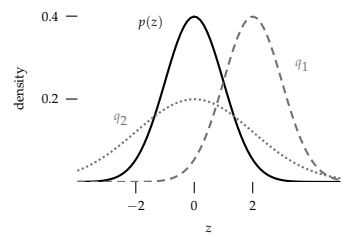


Figure 16: VAE architecture. Compare with Figure 15: the deterministic bottleneck is replaced by distribution parameters μ, σ and a stochastic sample. The loss combines reconstruction fidelity with KL regularisation toward the prior.

WE ENFORCE THAT BY ADDING A LOSS a KL divergence $\mathcal{L}_{\mathrm{KL}}(q_{\theta_E}(z|x) \| p(z))$, computed for each input separately. It measures how far that input's

encoder distribution $\mathcal{N}(\mu, \sigma^2)$ is from the prior $p(z) = \mathcal{N}(0, I)$. The KL between two Gaussians penalises two things: a mean μ that is far from the origin, and a variance σ^2 that deviates from one. So for each input, the KL term pulls the mean toward zero and the variance toward unity.



KL PENALTY. $p(z) = \mathcal{N}(0, 1)$ is the prior (solid). $q_1 = \mathcal{N}(2, 1)$ (dashed) is shifted; $q_2 = \mathcal{N}(0, 4)$ (dotted) is widened. Spreading mass (q_2, $\mathcal{L}_{\text{KL}}$=0.81) costs more than translating it (q_1, $\mathcal{L}_{\text{KL}}$=0.50, evaluated in Exercise 2).

IF THE KL TERM ACTED ALONE, every input would collapse onto the same distribution $\mathcal{N}(0, I)$, and the decoder could not tell any two inputs apart. The reconstruction term fights back: it needs different inputs to have distinguishable embeddings, so it pushes their means apart. Training finds a compromise. Each input gets a distribution that is close to the prior but not identical to it: the means spread out just enough for the decoder to reconstruct, and the variances stay close to one. When all these slightly shifted, roughly unit-variance Gaussians are added together, their aggregate approximates the prior. That is how the prior fills in the gaps between embeddings and gives the latent space its geometric structure. Together they turn the encoder into a feature extractor whose embeddings are both informative and geometrically well behaved: nearby embeddings correspond to similar inputs, and every region of the space is meaningful.

THE RESULT is a latent space where geometric proximity reflects semantic similarity.

VAE Variants

β-VAE[7] scales up the KL term to pressure the encoder to use the latent space efficiently. The loss reweights the with a single hyperparameter:

$$\mathcal{L}_\beta = \text{Reconstruction} + \beta \cdot \mathcal{L}_{\text{KL}}$$

VQ-VAE[8] replaces continuous latents with discrete codes from a learned codebook. Each encoder output z_e is snapped to the nearest codebook entry, giving reconstructional structure:

$$z_q = \arg \min_{e_k \in \text{codebook}} \|z_e - e_k\|$$

CONDITIONAL VAE[9] feeds a conditioning variable c, such as a class label, into both encoder and decoder. The model now learns conditional distributions, which enables directed generation:

$$q_{\theta_E}(z|x, c), \quad p_{\theta_D}(x|z, c)$$

[7] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner. β-VAE: Learning basic visual concepts with a constrained variational framework. In *International Conference on Learning Representations (ICLR)*, 2017. https://scholar.google.com/scholar?q=Higgins+2017+beta-VAE+learning+basic+visual+concepts

[8] A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. https://scholar.google.com/scholar?q=van+den+Oord+2017+neural+discrete+representation+learning+VQ-VAE

[9] K. Sohn, H. Lee, and X. Yan. Learning structured output representation using deep conditional generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015. https://scholar.google.com/scholar?q=Sohn+Lee+Yan+2015+learning+structured+output+conditional+generative

The Reparameterisation Trick

How do we now the distribution of the current embedding space? Our model now reconstructs data in two steps: draw z from the prior, then decode. To measure how well the model explains an observation x, we would need to sum over all possible embeddings z and check which ones decode close to x:

$$p_{\theta_D}(x) = \int p_{\theta_D}(x|z)\, p(z)\, dz$$

In words. The probability that the model generates x equals, summed over every possible latent z, the probability that the decoder produces x given that z, weighted by the prior probability of drawing that z in the first place. Each z is one possible explanation; the integral combines all of them.

Spoiler. We can't. This integral has no closed form and the latent space is too high-dimensional to sum over by brute force. Instead of summing over all of latent space, the encoder $q_{\theta_E}(z|x)$ output is used as an estimate, for each input x, where useful embeddings z for x live. We sample only from that region[10], turning an impossible integral into a practical expectation over a handful of samples.

[10] D. P. Kingma and M. Welling. Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*, 2014. https://scholar.google.com/scholar?q=Kingma+Welling+2014+Auto-Encoding+Variational+Bayes

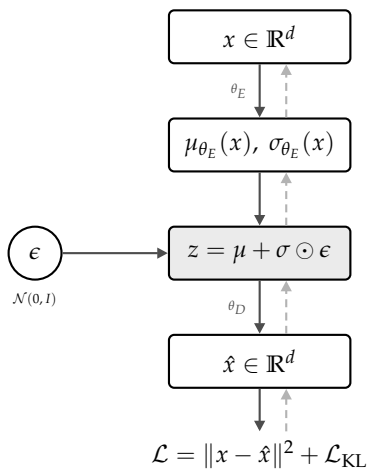


Figure 17: Reparameterised VAE or **Kingma's view**. Compare with Figure 16: the stochastic sample is replaced by $z = \mu + \sigma \odot \epsilon$ where ϵ carries the randomness and ,is drawn from a fixed distribution. Gradients (dashed) flow through μ and σ without passing through the random sample.

Designing the Bottleneck

THE LATENT DIMENSIONALITY k is the most consequential hyperparameter in a VAE. It controls the capacity of the information channel between encoder and decoder: how many bits of information about x can pass through z.

$$z \in \mathbb{R}^k$$

The bottleneck embedding. The dimensionality k controls how much information passes from encoder to decoder.

TOO FEW DIMENSIONS and the encoder cannot represent the factors of variation in the data. Reconstruction quality degrades because the embedding lacks the capacity to distinguish meaningfully different inputs. Two inputs that differ in important ways may map to nearly identical latent vectors, and the decoder produces a blurred average of what those embeddings could mean.

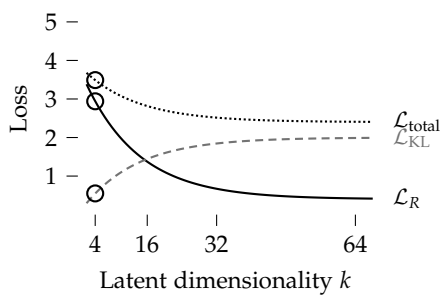


Figure 18: Bottleneck too small. At k=4, reconstruction loss is large ($\mathcal{L}_R \approx 2.9$) because the few active dimensions cannot encode all factors of variation. Each active dimension carries a lot of signal, but only a handful of them are paying KL ($\mathcal{L}_{\text{KL}} \approx 0.55$). The total (○ on the dotted curve) sits high because reconstruction error dominates.

TOO MANY DIMENSIONS and the model faces the opposite problem. With excess capacity, the encoder can encode each training point precisely without needing to discover shared structure. The KL term penalises unused dimensions toward the prior, but is blended out by the reconstruction success.

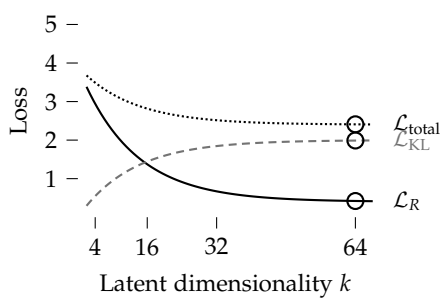


Figure 19: Bottleneck too large. At k=64, reconstruction loss has saturated near its floor ($\mathcal{L}_R \approx 0.42$) and the KL cost has saturated near its ceiling ($\mathcal{L}_{\text{KL}} \approx 1.99$): adding more dimensions changes neither term, because extra dimensions collapse to the prior and pay no KL. The total (○ on the dotted curve) sits at the plateau. The cost of k too large is therefore wasted parameters and slower training, not a higher loss.

THE INFORMATION BOTTLENECK PRINCIPLE[11] provides a theoretical lens. The optimal embedding retains all information in x that is relevant to reconstruction. The latent dimensionality k sets the ceiling

[11] N. Tishby, F. C. Pereira, and W. Bialek. The information bottleneck method. *arXiv preprint physics/0004057*, 2000. https://arxiv.org/abs/physics/0004057

on how much can be stored; the KL weight determines how aggressively the model compresses toward that ceiling to discarding weak representations.

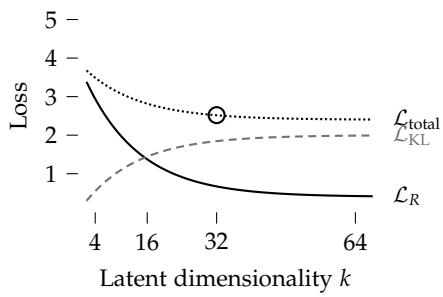


Figure 20: Bottleneck sizing trade-off. Reconstruction loss (solid) falls as k grows; the KL cost (dashed) rises until the data's information has been encoded, then plateaus as additional dimensions collapse to the prior. The total (dotted) is L-shaped: it drops sharply, then flattens. A good k sits near the elbow (○ at k=32), where the marginal gain per added dimension has fallen below what the extra parameters and training time cost.

ACTIVE UNITS. Inactive dimensions have collapsed to the prior, as there is no potential reconstruction gain contributed by their information. Monitoring the number of active units during training reveals how much of the latent capacity the model actually uses[12].

ACTIVE-UNIT CRITERION. A latent dimension j is called active if

$$\mathcal{L}_{\text{KL}}(q(z_j|x) \,\|\, p(z_j)) > \delta$$

for a meaningful fraction of the training data.

[12] Y. Burda, R. Grosse, and R. Salakhutdinov. Importance weighted autoencoders. In *International Conference on Learning Representations (ICLR)*, 2016. https://arxiv.org/abs/1509.00519

THIS READING ONLY HOLDS IN A PARTICULAR REGIME. The active-unit count is meaningful when k is rather large that some dimensions could afford to drop, and when the reconstruction loss is competitive with the KL pull. The diagnostic is really a measure of how many dimensions the reconstruction loss can keep alive against the KL pull. So cross-dimension activity comparisons are needed.

A PRACTICAL PROTOCOL for choosing k:

- Start with a generously large k. Adding capacity rarely hurts validation reconstruction; it just stops helping past the elbow.
- Train once and read off the effective capacity. The number of active dimensions is a direct estimate of the data's information ceiling.
- Use validation reconstruction, not training loss, as the model-selection signal to be sensitive to overfitting.

RULE OF THUMB. For input dimensionality d, a reasonable starting point is k on the order of $\sqrt{d}$ for tabular and pixel data. Text is a special case: vocabulary size makes d huge (30k+ tokens), but semantic information for sentence-level text VAEs, start at $k \approx 32-128$.

TWO EXCEPTIONS where the protocol does not apply. For visualisation, set k=2 or k=3 outright and accept the fidelity loss; the goal is a readable scatter, not optimal reconstruction. For very large models, where a single training run is expensive, pick a k from a comparable published architecture and skip the sweep entirely.

BEYOND DIMENSIONALITY, the structure of the prior itself shapes what the latent space can express. The standard choice $p(z) = \mathcal{N}(0, I)$ assumes the latent factors are independent and identically scaled. When the true factors of variation have different scales, correlations, or discrete structure, a mismatched prior forces the encoder to waste capacity compensating for the mismatch.

KL ANNEALING ramps the KL weight from zero over the first N epochs, letting the encoder learn useful representations before regularisation kicks in and prevents posterior collapse.

S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio. Generating sentences from a continuous space. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning (CoNLL)*, pages 10–21, 2016. https://arxiv.org/abs/1511.06349

Exploring the Latent Space

LATENT INTERPOLATION reveals the structure that the VAE has learned. Given two data points x_1 and x_2, encode, interpolate in the latent space, then decode each step:

$$
\begin{aligned}
z_1 &= \mu_{\theta_E}(x_1) \\
z_2 &= \mu_{\theta_E}(x_2) \\
z_t &= (1-t)\, z_1 + t\, z_2 \\
\hat{x}_t &= \theta_D(z_t)
\end{aligned}
$$

Because the latent space is regularised, the decoded path transitions smoothly between the two inputs[13]. Smooth interpolation is the simplest test of whether the encoder has learned a genuinely continuous feature space rather than a scattered collection of embeddings.

[13] T. White. Sampling generative networks. *arXiv preprint arXiv:1609.04468*, 2016. https://scholar.google.com/scholar?q=White+2016+Sampling+Generative+Networks

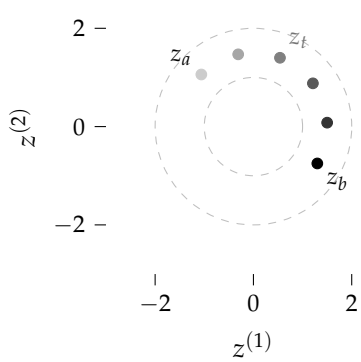


Figure 21: Latent interpolation. Two inputs encode to z_a and z_b. Linear interpolation (lerp, dashed) cuts through the low-density middle of the prior; in high dimensions this region is empty under $\mathcal{N}(0, I)$ and decoded midpoints look unnatural. Spherical linear interpolation (slerp, solid arc) follows the high-density shell at roughly $\|z\| \approx \sqrt{d}$, keeping every z_t on-distribution. Each waypoint is decoded to produce a smooth transition.

GENERATION is equally straightforward: sample from the prior and decode.

$$
\begin{aligned}
z &\sim \mathcal{N}(0, I) \\
\hat{x} &= \theta_D(z)
\end{aligned}
$$

The regularisation ensures that every region of the prior generates plausible data. The same property that makes the encoder a reliable feature extractor, a latent space with no dead zones, is what makes generation possible.

WELCOME TO GENERATIVE AI. A trained decoder paired with a tractable prior is, in essence, a generative model: draw a vector from a Gaussian, run it through the network, and a new sample falls out the other side. Everything that followed in the field, from diffusion models to large image and video generators, builds on the same trick of learning a mapping from a simple prior to a complex data distribution.

Examples & Exercises

THE EXERCISES START WITH A FEW NUMERICAL PASSES to fix the mechanics, then a set of concept and diagnostic prompts that target the key ideas of the chapter one at a time, and finally a hands-on notebook on MNIST. Worked solutions for the compute exercises are in the body; concept exercises carry the answer in a margin note. Try it yourself first, then check.

VAE LOSS BY THE NUMBERS. For a 1D VAE with encoder output μ=1, σ=1 on input x=3 with reconstruction $\hat{x}$=2.7, compute the total loss $\mathcal{L} = \mathcal{L}_R + \mathcal{L}_{\mathrm{KL}}$ using squared error and the Gaussian KL closed form $\frac{1}{2}(\mu^2 + \sigma^2 - \log\sigma^2 - 1)$.

RECONSTRUCTION LOSS.

$$\begin{aligned}\mathcal{L}_R &= (x - \hat{x})^2 \\ &= (3 - 2.7)^2 \\ &= 0.3^2 \\ &= 0.09\end{aligned}$$

KL TERM.

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}} &= \tfrac{1}{2}(\mu^2 + \sigma^2 - \log\sigma^2 - 1) \\ &= \tfrac{1}{2}(1 + 1 - \log 1 - 1) \\ &= \tfrac{1}{2}(1 + 1 - 0 - 1) \\ &= \tfrac{1}{2}(1) \\ &= 0.50\end{aligned}$$

TOTAL LOSS.

$$\begin{aligned}\mathcal{L} &= \mathcal{L}_R + \mathcal{L}_{\mathrm{KL}} \\ &= 0.09 + 0.50 \\ &= 0.59\end{aligned}$$

SECOND CONFIGURATION: μ=0, σ=1, x=3, $\hat{x}$=2.0.

$$\begin{aligned}\mathcal{L}_R &= (3-2.0)^2 \\ &= 1.00 \\ \mathcal{L}_{\mathrm{KL}} &= \tfrac{1}{2}(0+1-0-1) \\ &= 0 \\ \mathcal{L} &= 1.00+0 \\ &= 1.00\end{aligned}$$

The encoder matches the prior exactly so the KL drops to zero, but reconstruction more than triples. The first configuration achieves a lower total loss by spending some KL for a much better fit; that trade-off is what training resolves.

WHERE THE TRADE-OFF LIVES. Reconstruction is small in configuration one but the KL penalty is five times larger; in configuration two the KL is free but reconstruction dominates. Training finds the encoder that minimises the sum, not either term in isolation.

KL DIVERGENCE BY HAND. Compute $\mathcal{L}_{\mathrm{KL}}(\mathcal{N}(\mu,\sigma^2)\,\|\,\mathcal{N}(0,1)) = \frac{1}{2}(\mu^2+\sigma^2-\log\sigma^2-1)$ for three settings, and split each result into a shift cost $\frac{1}{2}\mu^2$ and a spread cost $\frac{1}{2}(\sigma^2-\log\sigma^2-1)$.

SETTING (I): μ=0, σ=1 (posterior matches the prior).

$$\begin{aligned}\tfrac{1}{2}\mu^2 &= 0 \\ \tfrac{1}{2}(\sigma^2-\log\sigma^2-1) &= \tfrac{1}{2}(1-0-1) \\ &= 0 \\ \mathcal{L}_{\mathrm{KL}} &= 0+0 \\ &= 0\end{aligned}$$

SETTING (II): μ=1, σ=1 (mean shifted, same spread).

$$\begin{aligned}\tfrac{1}{2}\mu^2 &= \tfrac{1}{2}\times 1 \\ &= 0.50 \\ \tfrac{1}{2}(\sigma^2-\log\sigma^2-1) &= \tfrac{1}{2}(1-0-1) \\ &= 0 \\ \mathcal{L}_{\mathrm{KL}} &= 0.50+0 \\ &= 0.50\end{aligned}$$

SETTING (III): μ=0, σ=2 (correct mean, too wide).

$$\begin{aligned}
\tfrac{1}{2}\mu^2 &= 0 \\
\tfrac{1}{2}(\sigma^2 - \log\sigma^2 - 1) &= \tfrac{1}{2}(4 - \log 4 - 1) \\
&= \tfrac{1}{2}(4 - 1.386 - 1) \\
&\approx 0.81 \\
\mathcal{L}_{\mathrm{KL}} &\approx 0 + 0.81 \\
&\approx 0.81
\end{aligned}$$

WHY THE ASYMMETRY. The shift cost is quadratic in μ. The spread cost is asymmetric: it grows roughly linearly as σ exceeds 1, but the $-\log\sigma^2$ term blows up as $\sigma \to 0$, so narrow posteriors are the most expensive of all. Setting (iii) costs more than (ii) because spreading mass away from the prior is harder than translating it.

β-VAE BALANCE PASS. Reuse the first configuration from *VAE loss by the numbers* ($\mathcal{L}_R$=0.09, $\mathcal{L}_{\mathrm{KL}}$=0.50) and compute the β-VAE loss $\mathcal{L}_\beta = \mathcal{L}_R + \beta \cdot \mathcal{L}_{\mathrm{KL}}$ for $\beta \in \{0.5, 1, 4\}$.

β=0.5.

$$\begin{aligned}
\mathcal{L}_{\beta=0.5} &= 0.09 + 0.5 \times 0.50 \\
&= 0.09 + 0.25 \\
&= 0.34
\end{aligned}$$

β=1 (STANDARD ELBO).

$$\begin{aligned}
\mathcal{L}_{\beta=1} &= 0.09 + 1 \times 0.50 \\
&= 0.09 + 0.50 \\
&= 0.59
\end{aligned}$$

β=4.

$$\begin{aligned}
\mathcal{L}_{\beta=4} &= 0.09 + 4 \times 0.50 \\
&= 0.09 + 2.00 \\
&= 2.09
\end{aligned}$$

The KL contribution to the total grows from 73% (β=0.5) to 96% (β=4). At β=4 the encoder has every incentive to shrink μ toward zero and σ toward one, at the cost of reconstruction. At β=0.5 the prior pull is relaxed and the encoder can spread out for sharper reconstructions. β=1 is the standard ELBO balance.

WHY β MATTERS. Training does not just evaluate this loss once; it follows the gradient. Whichever term dominates the loss dominates the gradient, and that is what shapes the final encoder.

REPARAMETERISATION. For μ=0.5, σ=1.2, and a sampled $\epsilon = -0.3$, compute the reparameterised sample $z = \mu + \sigma\epsilon$ and verify the gradients with respect to μ and σ.

FORWARD PASS.

$$\begin{aligned} z &= \mu + \sigma \cdot \epsilon \\ &= 0.5 + 1.2 \times (-0.3) \\ &= 0.5 - 0.36 \\ &= 0.14 \end{aligned}$$

GRADIENTS.

$$\begin{aligned} \frac{\partial z}{\partial \mu} &= \frac{\partial}{\partial \mu}(\mu + \sigma\epsilon) \\ &= 1 \\ \frac{\partial z}{\partial \sigma} &= \frac{\partial}{\partial \sigma}(\mu + \sigma\epsilon) \\ &= \epsilon \\ &= -0.3 \end{aligned}$$

Both gradients are finite scalars, so μ and σ get a clean signal from backpropagation; the randomness lives in ϵ, off the gradient path.

KL COST, BY INSPECTION. Without computing, rank the following encoder distributions by KL cost against the prior $\mathcal{N}(0, 1)$, lowest first.

- (i) μ=0, σ=1.
- (ii) μ=0, σ=0.3.
- (iii) μ=1.5, σ=1.
- (iv) μ=0, σ=3.

ANSWER. (i) < (iii) < (iv) < (ii). (i) matches the prior exactly so the KL is zero. (iii) only shifts the mean; the cost grows quadratically with μ. (iv) is too wide but the spread cost is roughly linear in σ^2. (ii) is too narrow, and the $-\log\sigma^2$ term blows up as $\sigma \to 0$, making narrow posteriors the most expensive.

AE VERSUS VAE: PREDICT THE EMBEDDING. You train a vanilla autoencoder and a VAE with k=2 on MNIST, both to convergence, and plot the validation embeddings.

- Which space concentrates encodings near the origin, and which spreads them arbitrarily across the plane?
- Does proximity in the latent space imply the inputs are visually similar? Is the answer the same for both models?

- If you pick a random point from a sparsely populated region and decode it, in which space is the result a plausible digit?
- Why k=2 and not k=10, given that MNIST has ten digit classes?

ANSWER. The AE places encodings wherever reconstruction loss is lowest; the layout is arbitrary, with no global structure and large unused regions. The VAE pulls every encoding toward the prior, so the cloud concentrates near the origin and fills the unit disc. Proximity reflects visual similarity in both, because the decoder is smooth, but only the VAE gives geometric position a global meaning: any z sampled from the prior decodes to something plausible, while a random point in an AE's empty region decodes to noise. k=2 is for visualisation; you plot $z^{(1)}$ against $z^{(2)}$ directly. At k=10 you would need t-SNE or UMAP to see the layout, and the differences you observe would be artefacts of the projection rather than of the AE-vs-VAE distinction.

PER-DIMENSION KL. For a k=4 VAE on a single input, the encoder outputs the parameters below (independent dimensions, prior $\mathcal{N}(0,1)$ per dim). Compute $\mathcal{L}_{\mathrm{KL}}^{(j)} = \frac{1}{2}(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)$ for each dimension and the total $\mathcal{L}_{\mathrm{KL}} = \sum_j \mathcal{L}_{\mathrm{KL}}^{(j)}$, then label each dimension active or collapsed.

DIM 1: μ_1=1.5, σ_1=0.5.

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}}^{(1)} &= \tfrac{1}{2}(1.5^2 + 0.5^2 - \log 0.5^2 - 1)\\ &= \tfrac{1}{2}(2.25 + 0.25 + 1.386 - 1)\\ &= \tfrac{1}{2}(2.886)\\ &\approx 1.44\end{aligned}$$

DIM 2: μ_2=0, σ_2=1.

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}}^{(2)} &= \tfrac{1}{2}(0 + 1 - 0 - 1)\\ &= 0\end{aligned}$$

DIM 3: μ_3=0.2, σ_3=0.9.

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}}^{(3)} &= \tfrac{1}{2}(0.2^2 + 0.9^2 - \log 0.9^2 - 1)\\ &= \tfrac{1}{2}(0.04 + 0.81 + 0.211 - 1)\\ &= \tfrac{1}{2}(0.061)\\ &\approx 0.03\end{aligned}$$

DIM 4: $\mu_4 = -0.8$, σ_4=0.7.

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}}^{(4)} &= \tfrac{1}{2}((-0.8)^2 + 0.7^2 - \log 0.7^2 - 1)\\ &= \tfrac{1}{2}(0.64 + 0.49 + 0.713 - 1)\\ &= \tfrac{1}{2}(0.843)\\ &\approx 0.42\end{aligned}$$

TOTAL AND VERDICT.

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}} &= 1.44 + 0 + 0.03 + 0.42\\ &\approx 1.89\end{aligned}$$

Dimensions 1 and 4 are active: large per-dim KL, σ well below 1, mean clearly away from zero. Dimension 2 has collapsed to the prior. Dimension 3 is borderline: nearly at the prior, contributing almost no information.

WHAT THE PER-DIM KL TELLS YOU. Posterior collapse is a per-dimension phenomenon: a single VAE can have some dimensions active and others collapsed in the same training run. Summing the KL hides that detail; reading it per dimension exposes which axes the encoder uses and which it has given up on.

COUNTING ACTIVE UNITS. A VAE with k=8 reports the following per-dimension posterior statistics, averaged over the validation set: the squared mean $\overline{\mu_j^2}$, the variance σ_j^2, and the resulting per-dimension KL $\mathcal{L}_{\mathrm{KL}}^{(j)} = \frac{1}{2}(\overline{\mu_j^2} + \sigma_j^2 - \log \sigma_j^2 - 1)$.

Dim j	1	2	3	4	5	6	7	8
$\overline{\mu_j^2}$	1.50	0.01	0.80	0.00	1.20	0.00	0.60	0.70
σ_j^2	0.12	0.97	0.31	0.99	0.18	1.00	0.41	0.95
$\mathcal{L}_{\mathrm{KL}}^{(j)}$	1.37	0.01	0.64	0.00	1.05	0.00	0.45	0.35

Table 4: Per-dimension posterior statistics for a k=8 VAE on the validation set. Inactive dimensions have $\overline{\mu_j^2} \approx 0$ and $\sigma_j^2 \approx 1$, so their KL contribution falls to zero and they have collapsed back to the prior. Active dimensions either spread the means (large $\overline{\mu_j^2}$), tighten the variance (small σ_j^2), or both, and pay a positive KL for the information they carry.

How many dimensions are doing real work? Which ones can you remove without losing reconstruction quality? If you re-trained at k=4, would you expect the same number of active dimensions?

ANSWER. Five active dimensions: 1, 3, 5, 7, 8, identifiable by $\mathcal{L}_{\mathrm{KL}}^{(j)}$ well above zero. Dimensions 2, 4, 6 have $\mathcal{L}_{\mathrm{KL}}^{(j)} \approx 0$ and have collapsed to the prior; dropping them costs nothing. Note that dim 8 looks borderline by variance alone ($\sigma_8^2 = 0.95$), but its mean spread $\overline{\mu_8^2} = 0.70$ keeps it carrying information, which is why the KL is the cleaner diagnostic. Re-training at k=4 should still yield about four to five active units, because the active count is set by the data's information content, not by k once k is past the elbow.

DIAGNOSE THE TRAINING RUN. Three runs of the same VAE on the same data give different end-of-training behaviour. For each, name what is happening and what you would change.

Run	$\mathcal{L}_R$	$\mathcal{L}_{\mathrm{KL}}$	**Reconstructions**
A	high	≈ 0	blurry mean digit
B	low	high	sharp but unrelated to input
C	low	moderate	sharp and semantically sound

Table 5: Three end-of-training profiles for the same VAE on the same data. One is healthy training, one is posterior collapse, and one is a memorising encoder that bypasses the prior.

ANSWER. **A** is posterior collapse: KL near zero means the encoder ignores x, so the decoder outputs the data mean. Lower β or anneal it from zero. **B** has the encoder bypassing the prior to memorise inputs (high KL, low reconstruction, but sharp outputs that do not generalise to sampled z). Raise β or check for an over-parameterised encoder. **C** is what you want: both terms contribute and reconstructions track the inputs.

KL ANNEALING BY THE NUMBERS. A linear-ramp schedule sets

$$\beta(t) = \min\left(\tfrac{t}{10}, 1\right)$$

so β rises from 0 at epoch 0 to 1 at epoch 10 and stays at 1 afterwards. Assume that on a fixed validation batch the encoder reports $\mathcal{L}_R$=0.4 and $\mathcal{L}_{\mathrm{KL}}$=1.2 throughout training (the values themselves do not change, only the weight). Compute the total loss $\mathcal{L}_\beta(t) = \mathcal{L}_R + \beta(t) \cdot \mathcal{L}_{\mathrm{KL}}$ at epochs $t \in \{1, 5, 10, 20\}$.

EPOCH 1: β=0.1.

$$\begin{aligned}\mathcal{L}_\beta(1) &= 0.4 + 0.1 \times 1.2 \\ &= 0.4 + 0.12 \\ &= 0.52\end{aligned}$$

EPOCH 5: β=0.5.

$$\begin{aligned}\mathcal{L}_\beta(5) &= 0.4 + 0.5 \times 1.2 \\ &= 0.4 + 0.60 \\ &= 1.00\end{aligned}$$

EPOCH 10: β=1.

$$\begin{aligned}\mathcal{L}_\beta(10) &= 0.4 + 1 \times 1.2 \\ &= 1.60\end{aligned}$$

EPOCH 20: β=1 (CLAMPED).

$$\mathcal{L}_\beta(20) = 1.60$$

At which epoch does the KL contribution first match the reconstruction term, and what would $\mathcal{L}(t)$ look like if you set β=1 from the start?

WHEN THE TERMS MATCH. $\beta(t) \cdot \mathcal{L}_{\mathrm{KL}} = \mathcal{L}_R$ when $\beta(t) = 0.4/1.2 \approx 0.33$, i.e. around epoch 3 on this schedule. Without the ramp, the KL would dominate from epoch 1, giving the encoder every incentive to ignore x before it has learned a useful representation; the warm-up window buys the encoder time to first encode information, then take the regularisation pressure.

READ THE OBSERVATIONS FROM THE EMBEDDING SPACE EXPLORATION. You linearly interpolate between two encoded digits in latent space and decode each step. For each described path, name what the latent space tells you.

- Path 1: a smooth morph from a 3 into an 8, passing through plausible intermediate shapes.
- Path 2: the 3 stays fixed for half the path, then suddenly jumps to the 8.
- Path 3: the path passes through several unrelated digit shapes (a 2, then a 5) before reaching the 8.

ANSWER. Path 1 indicates a well-organised latent space: nearby points decode to similar digits and the regulariser has filled the region between the two encodings. Path 2 means the encoder placed the two digits far apart with empty space in between; the prior has not pulled them close enough. Path 3 means the line crosses other class regions; that is geometrically honest but suggests the chosen direction is not a meaningful axis.

AN MNIST SUBSET TO EXPLORE. You are given the same thousand handwritten digits as in the previous chapter. Implement three pieces from scratch in numpy: a linear autoencoder, the closed-form KL divergence to a standard normal prior, and the reparameterisation trick $z = \mu + \sigma \odot \varepsilon$. Sweep k on the linear AE and inspect the reconstruction curve, then compare precomputed AE and VAE embeddings at k=2 and at $k \in \{2, 8, 32\}$. Optionally lift a k=16 VAE: bar-chart its per-dimension KL using your own implementation, and decode an interpolation between an encoded 0 and an encoded 1.

CODE is provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

1 000 MNIST digits
784 pixels each
ten classes

implement: linear AE, KL divergence, reparameterisation trick

WHAT TO OBSERVE. The linear AE elbow matches the PCA reconstruction curve, confirming that a tied-weight linear AE is PCA in disguise. The AE and VAE scatters at k=2 differ in geometry: AE encodings spread arbitrarily with large empty regions, VAE encodings concentrate near the origin and fill the unit disc. Active units saturate well below the chosen k. The interpolation passes through plausible intermediate digits, showing that the latent space is genuinely continuous.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- What does the autoencoder bottleneck force the encoder to discover, and how does it relate to the PCA subspace?
- Why does a deterministic autoencoder fail as a generative model?
- In the Gaussian KL, why is a narrow posterior more expensive than a shifted or widened one?
- When would you reach for β-VAE, VQ-VAE, or a conditional VAE over the plain VAE?
- Why does the reparameterisation trick work, and what does it exploit about the Gaussian?
- What is posterior collapse, and which design lever brings it on most often?
- How does the active-unit count tell you whether k is set correctly?
- Why does sampling $z \sim \mathcal{N}(0, I)$ and decoding produce plausible new data in a VAE but not in a plain autoencoder?

RECAP of Key Concepts:

- Autoencoders learn a nonlinear θ_E, θ_D through a bottleneck; linear AE recovers PCA, nonlinear goes beyond it.
- VAEs replace the point embedding with $q(z|x)$ regularised toward a prior, giving a continuous, generative latent space.
- The Gaussian KL trades a quadratic shift cost against an asymmetric spread cost; narrow posteriors cost the most.
- β-VAE buys disentanglement, VQ-VAE buys discrete codes, conditional VAE buys controlled generation.
- The reparameterisation trick $z = \mu + \sigma \odot \varepsilon$ keeps the sample differentiable in μ and σ.
- Active-unit count diagnoses bottleneck use; KL warm-up is the standard fix for posterior collapse.
- Sampling $z \sim p(z)$ and decoding generates new data; smooth interpolation tests latent continuity.

THE ROOT OF MODEL BLINDNESS. Any model can tell you what it has learned, but not how much to trust what it says about inputs it has never seen.

UNSUPERVISED DEEP LEARNING is at root the project of modelling what counts as familiar, and the VAE does this directly: low reconstruction error and high prior likelihood under $p(z)$ mean an input resembles the training data. Yet every model trained on finite data, generative or not, has a blind spot for inputs outside its experience: a VAE's error rises on unfamiliar inputs and a classifier's softmax stays peaked on adversarial ones, but neither flags the gap in any principled way.

UNCERTAINTY ESTIMATION turns familiarity into a principled tool, and it is itself an unsupervised problem: knowing where a model's knowledge ends depends on the learned distribution of inputs, not on a held-out label. The next chapter develops model-agnostic methods for separating irreducible noise from reducible ignorance, and uses them for anomaly detection, out-of-distribution flagging, and graceful degradation in deployment.

TEASER. How does a model trained without labels know when a new input falls outside the structure it has learned?

FEEDBACK.

Index

autoencoder, 35, 40, 42

β-VAE, 43
bottleneck design, 45

computational complexity
  UMAP, 26
conditional VAE, 43
covariance matrix, 19
cross-entropy, 26
crowding problem, 25
curse of dimensionality, 15, 18

dimensionality reduction, 15

eigenvalue, 20
eigenvector, 20
ELBO, 35, 42
embedding, 24
evidence lower bound, 42
examples, 27, 48
exercises, 27, 48
explained variance, 22

feature selection, 18

Gaussian kernel, 24
generation, 47
gradient descent, 25

hyperparameter selection, 24

ICA, 23
intrinsic dimensionality, 15

Johnson-Lindenstrauss lemma, 18

KL divergence, 24

Lagrange multiplier, 20
latent dimensionality, 45
latent interpolation, 47
latent variable model, 35
license, 2

manifold, 19

non-Gaussianity, 23

PCA, 15, 19
perplexity, 24

recap, 32, 56
reconstruction error, 23
reconstruction loss, 40
reflection, 32, 56
reparameterisation trick, 44
representation learning, 35

scree plot, 22
spectral embedding, 26
statistical independence, 23
Student-t kernel, 24
SVD, 23

t-SNE, 15, 24

UMAP, 15, 26

VAE, 35, 42
variance, 18
Variational Autoencoder, 35
variational inference, 35
visualization, 24
VQ-VAE, 43